



南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch71 键值数据库

Software Institute
Nanjing University
Bei Jia

键值数据库 1

- 键值数据库(key-value store)是一张简单的哈希表(hash table), 主要用在所有数据库访问均通过主键(primary key)来操作的情况下。
 - 可把此表想象成传统的“关系” 该关系有两列: ID与NAME
 - ID列代表关键字, NAME列存放值。NAME列仅能存放String型的数据。
 - 应用程序可提供ID及VALUE值, 并将这一键值对持久化
 - 假如ID已存在, 就用新值覆盖当前值, 否则就新建一条数据。

Oracle	Riak
database instance	Riak cluster
table	bucket
row	key-value
rowid	key

键值数据库 2

- 键值数据库是最简单的NoSQL数据库。
- 客户端可以根据键查询值，设置键所对应的值，或从数据库中删除键。
 - “值”只是数据库存储的一块数据而已，它并不关心也无需知道其中的内容
 - 应用程序负责理解所存数据的含义。
- 由于键值数据库总是通过主键访问，所以它们一般性能较高，且易于扩展。
- 流行的键值数据库有: Riak、Redis（数据结构服务器）、Memcached DB及其变种、Berkeley DB、HamsterDB (尤其适合嵌入式开发)、 Amazon DynamoDB (不开源)和Project Voldemort (Amazon DynamoDB的开源实现)。

数据结构服务器

- 在键值数据库中，所存储的聚合不一定是领域对象 (domain object)，也可以拥有通用数据结构
- Redis能够存储list、set、hash 等数据结构，可以支持“获取某个范围内的数值”(range)、“求差集”(diff)、“求并集”(union)、“求交集”(intersection) 等操作
- 这些功能使数据库的用途变得比标准键值数据库更多

单一存储区 1

- “存储区”(bucket)用于区隔关键字的一种手段, 可以将其视为存放关键字所用的“平坦命名空间”(flat namespace)
- 使用单一存储区, 把所有数据都放在一个对象里, 并将其存入单一的存储区中

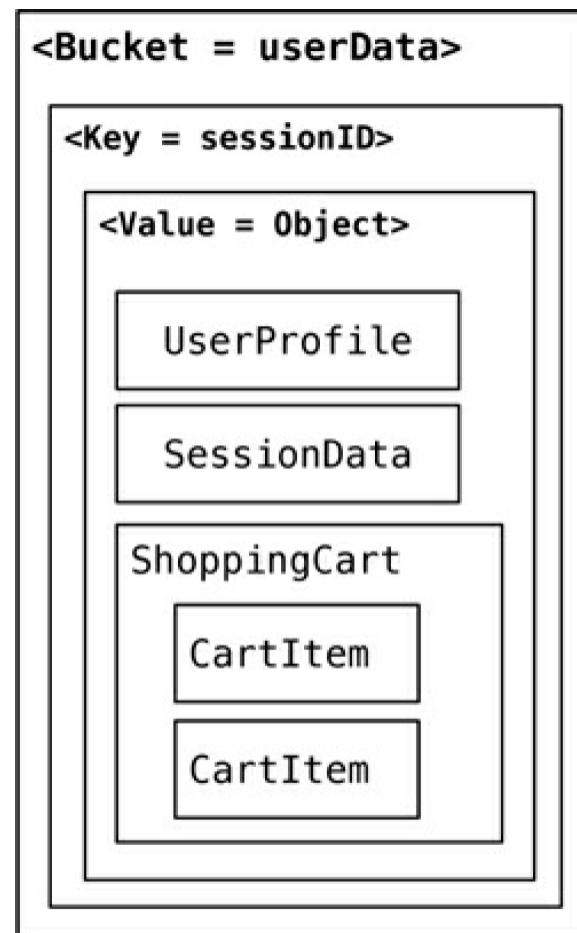


Figure 8.1. Storing all the data in a single bucket

单一存储区存储区 2

- 将各类对象(也就是聚合)全部存放在一个“存储区”中，其缺点是：“存储区”中可能要存放类型不同的多个聚合，这增加了关键字冲突的几率
- 还有一种办法是把对象名放在键名后面，例如：
288790b8a421_userProfile，这样就可用它查出所需的单个对象了

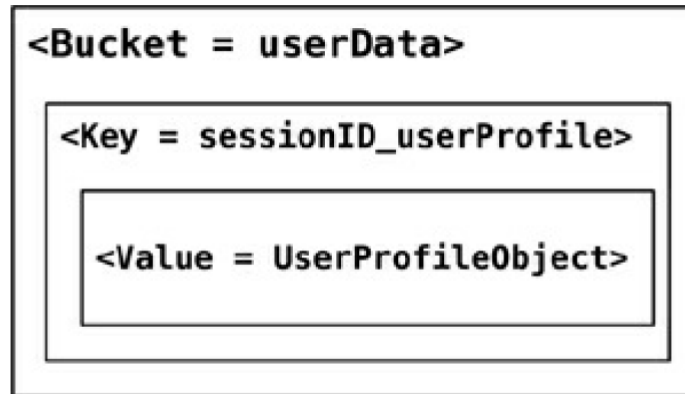


Figure 8.2. Change the key design to segment the data in a single bucket.

领域存储区

- “领域存储区”(domain bucket)来存放特定数据
- 客户端驱动程序可以对其执行“序列化”(serialization)与“反序列化”(deserialization) 操作
- 将跨越多个“存储区”的数据分割成对象，将之存放在“领域存储区”或不同的“存储区”中，这样一来，无需改变关键字的命名方式，即可读出所需对象
- 存放表达相同含义的不同聚合方案，以应对多种不同应用的需求
 - 效率及数据不一致性问题

一致性 1

- 只有针对单个键的操作才具备“一致性”，因为这种操作只可能是“获取”、“设置”或“删除”。
 - 由于数据库无法侦测数值改动，“乐观写入”(optimistic write)功能的实现成本太高。
- 分布式键值数据库，用“最终一致性模型”实现“一致性”。
 - 两种解决“更新冲突”的办法：
 - ◆ 采纳新写入的数据而拒绝旧数据
 - ◆ 将两者(或存在冲突的所有数据)返回给客户端，令其解决冲突

一致性 2

- 在创建“存储区”时设置与一致性有关的选项。
 - 若想提高数据一致性, 可以规定: 执行完写入操作后, 只有当存放此数据的全部节点一致将其更新, 才认定该操作生效。
 - ◆ 显然降低了集群的写入效率
 - 若想提高写入冲突或读取冲突的解决速度, 可在创建“存储区”时设置为数据库接纳最新的写入操作, 而不再创立“旁系记录”(sibling)。

事务

- 不同类型的键值数据库，其“事务”规范也不同，实现“事务”的方式各异。一般说来，无法保证写入操作的“一致性”。
- Riak在调用写入数据的API时，它使用W值与复制因子来实现“仲裁”。
 - 假设某个集群的复制因子是5，而W值为3。
 - 在写入数据时，必须有至少3个节点汇报其写入操作已顺利完成，数据库才会认为此操作执行完毕。
 - 由于N等于5而W是3，所以集群在两个节点($N-W=2$) 故障时仍可执行写入操作，不过，此时我们无法从那些发生故障的节点中读取某些数据。

查询功能

- 所有键值数据库都可以按关键字查询。它们的查询功能基本上仅限于此。
- 如果希望根据“值列”(value column)的某些属性来查询，那么无法用数据库完成此操作
 - 应用程序需要自己读出值，并判断其属性是否符合查询条件。
- 如果不知道关键字该怎么办？
 - 大部分数据库都不提供全部主键列表，即便提供了，获取关键字列表并查询其值的操作也很烦琐
 - 某些键值数据库支持数值搜寻，以解决此问题
- 通过API、HTTP（浏览器、Curl等），操作键值数据库

键名的设计

- 使用键值数据库时，通过某种算法生成键
 - 使用用户信息(例如ID、电子邮件地址等)、时间戳等值，生成键
- 键值数据库非常适合保存会话(用会话ID作为键)、购物车数据、用户配置等信息

数据结构

- 键值数据库并不关心键值对里的值。它可以是二进制块、文本、JSON、XML等。
- 可在HTTP请求中用Content-Type指定数据类型
 - 实质上是由应用判定其内容

可扩展性 1

- 很多键值数据库都可用“分片”技术扩展。采用此技术后，键的名字就决定了负责存储该键的节点。
 - 假设按照键名的首字母“分片”。如果键名是f4b19d79587d，那么由于其首字母为f,所以存放它的节点就与存放ad9c7a396542这个键的节点不同。
 - 当集群中的节点数变多时，这种“分片”设定可提高效率。
- “分片”也会引发某些问题。假如存放首字母为f的键所用的那个节点坏了，那么其上的数据将无法访问，而且也不能再写入其他键名首字母为f的新数据了。

可扩展性 2

- 可以控制“CAP定理”中的参数: N (存放键值对的副本节点数)、R (顺利完成读取操作所需的最小节点数)和W (顺利完成写入操作所需的最小节点数)。
 - 假设集群有5个节点。将N设为3, 意思就是所有数据都至少要复制到3个节点中, 将R设为2, 意思是GET请求要有两个节点应答, 才能成功, 将W设为2,意思是PUT请求必须写入两个节点, 才算执行完毕。
- 可以利用这些设置来微调读取及写入操作所能容忍的故障节点数。应该按照应用的需要来改变这些值, 以提升数据库的“可读能力”(read availability) 及“可写能力”(write availability)。通常应该根据“一致性”需求来确定W值。
- 创建“存储区”时可设定上述各参数的默认值。

适用案例-存放会话信息

- 通常来说，每一次网络会话都是唯一的，所以分配给它们的sessionid值也各不相同。
- 如果应用程序原来要把sessionid存在磁盘上或关系型数据库中，那么将其迁移到键值数据库之后，会获益良多
 - 因为全部会话内容都可以用一条PUT请求来存放，而且只需一条GET请求就能取得。
 - 由于会话中的所有信息都放在一个对象中，所以这种“单请求操作”(single-request operation) 很迅速。

适用案例-用户配置信息

- 用户配置信息，几乎每位用户都有userId、username或其他独特的属性，而且其配置信息也各自独立，诸如语言、颜色、时区、访问过的产品等。
- 这些内容可全部放在一个对象里，以便只用一次GET操作即获取某位用户的全部配置信息。
- 同理，产品信息也可如此存放。

适用案例-购物车数据

- 购物车数据，电子商务网站的用户都与其购物车相绑定。
- 由于购物车的内容要在不同时间、不同浏览器、不同电脑、不同会话中保持一致，所以可把购物信息放在value属性中，并将其绑定到userid这个键名上。

不适用场合

- 数据间关系

- 如果要在不同数据集之间建立关系，或是将不同的关键字集合联系起来，那么即便某些键值数据库提供了“链接遍历”等功能，它们也不是最佳选择。

- 含有多项操作的事务

- 如果在保存多个键值对时，其中有一个关键字出错，而又需要复原或回滚其余操作，那么键值数据库就不是最好的解决方案。

- 查询数据

- 如果要根据键值对的某部分值来搜寻关键字，那么键值数据库就不是很理想了。我们无法直接检视键值数据库中的值，除非使用某些“检索引擎”(indexing engine)。

- 操作关键字集合

- 由于键值数据库一次只能操作一个键，所以它无法同时操作多个关键字。假如需要操作多个关键字，那么最好在客户端处理此问题。



南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch72 文档数据库

Software Institute
Nanjing University
Bei Jia

文档数据库 1

- “文档”(document)是文档数据库中的主要概念。
 - 其格式可以是XML、JSON、BSON等。
 - 文档具备自述性(self-describing), 呈现分层的树状数据结构(hierarchical tree data structure), 可以包含映射表、集合和标量值。
- 文档彼此相似, 但不必完全相同。文档数据库所存放的文档, 就相当于键值数据库所存放的“值”。
- 文档数据库可视为其值可查的键值数据库。

Oracle	MongoDB
database instance	MongoDB instance
schema	database
table	collection
row	document
rowid	_id
join	DBRef

文档数据库 2

- 文档数据库中，放在同一“集合”内的各文档的“数据模式”(the schema of the data)可以不同
 - 关系型数据库中，表格中每行数据的模式都要相同。
- 文档中可以嵌套数组等基本数据类型，也可以将“子文档”(child document) 以“子对象”(subobject) 的形式嵌入主文档。
- 由于没有“数据模式”约定，文档数据库的文档中无需空属性，若其中不存在某属性，就假定该属性值未设定或与此文档无关。向文档中新增属性时，既无需预先定义，也不用修改已有文档内容。
 - 关系型数据库中，需要定义表中的每一列，而且若某条记录中的某列没有数据，则要将其留空(empty) 或设为null。
- 流行的文档数据库有: MongoDB、 CouchDB、 Terrastore、 OrientDB、 RavenDB和Lotus Notes。

一致性

- 通过配置“副本集”(replica set) 实现“复制”， 以提供较高的“可用性”
- 规定写入操作必须等待所写数据复制到全部或是给定数量的从节点之后， 才能返回。从而指定数据库的“一致性”强度。
 - 在只有一台服务器时如果指定w为“majority”， 那么写入操作立刻就会返回， 因为总共只有一个节点。
 - 假设“副本集”中有三个节点， 则写入操作必须在至少两个节点上执行完毕， 才会视为成功
 - 提升w值可以增强“一致性”， 但是会降低写入效率， 因为写入操作必须在更多的节点上完成才行。
- 也可以增加“副本集”的读取效率:设置slaveOk选项之后， 就可以于从节点中读取数据了。
- 参数既可设置到整个“连接”、“数据库”、“集合”之上， 也可针对每项操作独立设置。

事务

- 大多数文档数据库通常没有事务机制:其写入操作要么成功, 要么失败。
- “单文档级别”(single-document level)的“事务”叫做“原子事务”(atomic transaction)。
- 可以用不同级别的WriteConcern参数来确保各种安全级别的写入操作
 - 在默认情况下, 所有写入操作都将顺利执行。
 - 以WriteConcern.REPLICAS_SAFE为参数写入,即可确保该操作至少要写入两个节点才算成功。
 - 在写日志条目(log entry)时, 就可使用最低的安全级别, 也就是WriteConcern.NONE。

可用性 1

- 文档数据库可以用主从式数据复制技术来增强“可用性”。多个节点都保有同一份数据，即便主节点故障，客户端也依然能获取数据。应用程序代码一般不需检测主节点是否可用。
- MongoDB通过“副本集”实现“复制”，以提供较高的“可用性”。副本集中至少有两个节点参与“异步主从式复制”(asynchronous master-slave replication).
 - “副本集”通常用于处理“数据冗余”(data redundancy)、“自动故障切换”(automated failover)、“读取能力扩展”(read scaling)、“无需停机的服务器维护(server maintenance without downtime)和“灾难恢复”(disaster recovery)等事项。
- 应用程序的写入或读取操作都针对主节点。建立连接后，应用程序只需要同“副本集”中的一个节点相连即可(是不是主节点无所谓)，数据库会自动找到其余节点。若主节点故障，则数据库驱动会同“副本集”中新选出的主节点联系。应用程序不用处理通信错误，也无需干预主节点的选拔准则。

可用性 2

- 副本集在其内部选举“主”(master)节点，或“主要”(primary)节点。假定所有节点投票权相同，其中某些节点可能会因为距离其他服务器较近，或具有更多运行内存(RAM)等因素而获得更多选票。用户也可以为节点指定一个值在0 ~ 1000之间的优先级(priority)来影响选举过程。
- 所有请求都由主节点处理，而其数据会复制到从节点。若主节点故障，则“副本集”中剩下的节点就会在其自身范围内选出新的主节点，所有后续请求就交由新的主节点处理，从节点也开始从新的主节点处获取数据。
- 当原来的主节点从故障中恢复时，它会作为从节点重新加入，并获取全部最新数据，以求与其他节点一致。

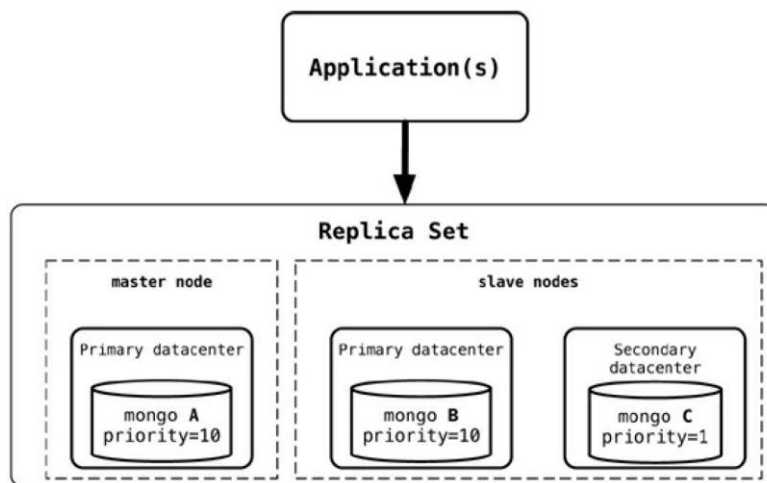


Figure 9.1. Replica set configuration with higher priority assigned to nodes in the same datacenter

查询功能

- 文档数据库可以查询文档中的数据，而不用像键值数据库那样，必须根据关键字获取整个文档，然后再检视其内容。
- CouchDB:可用“物化视图”(materialized view)或“动态视图”(dynamic view)实现复杂的文档查询。
- MongoDB支持一种JSON格式的查询语言
 - 由于文档是“聚合对象”(aggregated object)，所以用带子对象的字段查询待匹配的文档非常方便。

可扩展性 1

- 在不将数据库进行迁移的前提下，向其中新增节点或修改其内容。
- 增加更多的“读取从节点”(read slave)，将读取操作导引至从节点上，这样就可以扩展数据库应对频繁读取的能力了。
 - 假设某个应用程序的读取操作很频繁，可向“副本集”中加入更多从节点，并在执行读取操作时设定slaveOk标志，以提升集群的读取能力。完成读取操作的横向扩展。

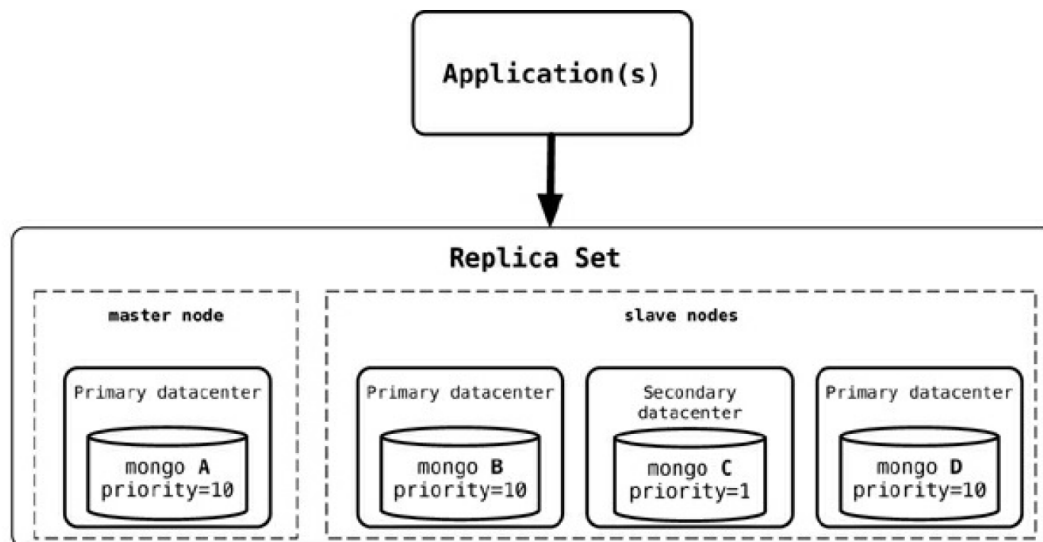


Figure 9.2. Adding a new node, mongo D, to an existing replica-set cluster

可扩展性 2

- 如果想扩展写入能力，可以把数据“分片”。
 - “分片”与关系型数据库的“分区”类似，
 - ◆ “分区”是根据某列的值，例如状态或年份，将数据分割开。关系型数据库的“分区”通常位于同一节点，所以客户端应用程序只查询“基表”(base table)就好，不需查询某个特定分区，关系型数据库会根据查询内容搜索适当的分区并返回数据。
 - ◆ “分片”操作也根据特定字段来划分数据，然而那些数据要移动到不同的Mongo节点中。为了让各“分片”的负载保持均衡，需要在节点之间动态转移数据。向集群中新增更多节点，并提高可写入的节点数，就能横向扩展其写入能力。
- “分片”的关键字很重要。
 - 按照客户名字(first name)来分隔，可确保将数据平衡地散布在各个“分片”上，以获得较好的写入效率。
 - 如果想把“分片”放在距离用户近的地方，那么可以以用户位置来分片。按客户位置分片时，美国东海岸的全部用户数据都会放在居于东海岸的“分片”中，而所有西海岸的用户数据则将放在位于西海岸的“分片”中。

可扩展性 3

- 可以把每个“分片”都做成“副本集”，以提高其读取效率。
- 如果向已有的“分片集群”(sharded cluster)中再加一个新分片”，就可以把原来分布在3个“分片”中的数据打散到4个“分片”中。
 - 在转移数据与底层设施重构的全过程中，虽说集群为了重新平衡“分片”负载而传输大量数据时性能也许会下降，但是应用程序却无需停止工作。

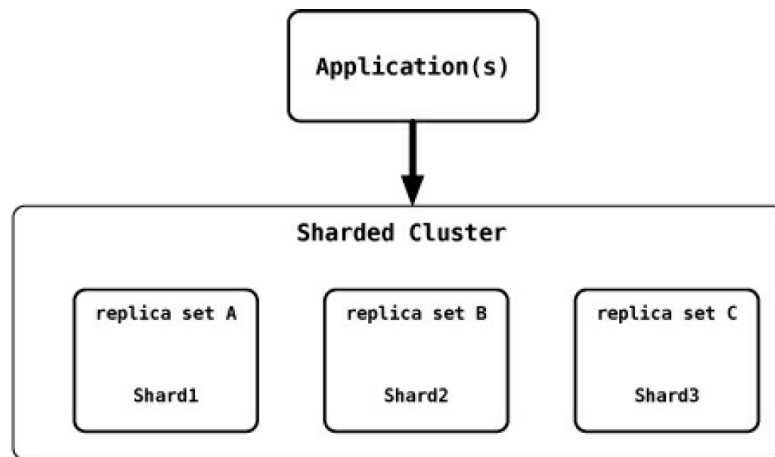


Figure 9.3. MongoDB sharded setup where each shard is a replica set

适用案例-事件记录

- 在企业级解决方案中，许多不同的应用程序都需要记录事件。应用程序对事件记录各有需求。
- 文档数据库可以把所有这些不同类型的事件都存起来，并作为事件存储的“中心数据库”(central data store)使用。
- 如果事件捕获的数据类型一直在变，那么就更应该用文档数据库了。
- 可以按照触发事件的应用程序名“分片”，也可以按照order_processed或customer_logged等事件类型“分片”

适用案例-其他

- 内容管理系统及博客平台

- 由于文档数据库没有“预设模式”(predefined schema)，而且通常支持JSON文档，所以它们很适合用在“内容管理系统”(content management system)及网站发布程序上，也可以用来管理用户评论、用户注册、用户配置和面向Web文档(web-facing document)。

- 网站分析与实时分析

- 文档数据库可存储实时分析数据。由于可以只更新部分文档内容，所以用它来存储“页面浏览量”(page view)或“独立访客数”(unique visitor)会非常方便，而且无需改变模式即可新增度量标准。

- 电子商务应用程序

- 电子商务类应用程序通常需要较为灵活的模式，以存储产品和订单。同时，它们也需要在不做高成本数据库重构及数据迁移的前提下进化其数据模型。

不适用场合

- 包含多项操作的复杂事务
 - 文档数据库也许不适合执行“跨文档的原子操作”(atomic cross-document operation)，虽然像RavenDB等文档数据库其实也支持此类操作。
- 查询持续变化的聚合结构
 - 灵活的模式意味着数据库对模式不施加任何限制。数据以“应用程序实体”(application entity)的形式存储。
 - 如果要即时查询这些持续改变的实体，那么所用的查询命令也得不停变化(用关系型数据库的术语讲，就是用JOIN语句将数据表按查询标准连接起来时，待连接的表一直在变)。
 - 由于数据保存在聚合中，所以假如聚合的设计持续变动，那么就需要以“最低级别的粒度”(lowest level of granularity)来保存聚合了，这实际上就等于要统一数据格式了。在这种情况下，文档数据库也许不合适。



南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch73 列族数据库

Software Institute
Nanjing University
Bei Jia

列族数据库

- 列族数据库, 可以存储关键字及其映射值,并且可以把值分成多个列族, 让每个列族代表一张数据映射表(map of data)。
- Cassandra是一款流行的列族数据库, 采用对等集群, 能快速执行跨集群写入操作并易于对此扩展。
- 此外还有HBase、Hypertable 和 Amazon DynamoDB等其他产品。

RDBMS	Cassandra
database instance	cluster
database	keyspace
table	column family
row	row
column (same for all rows)	column (can be different per row)

列族数据库

- 列族数据库将数据存储在列族中，而列族里的行则把许多列数据与本行的“行键”(row key)关联起来。

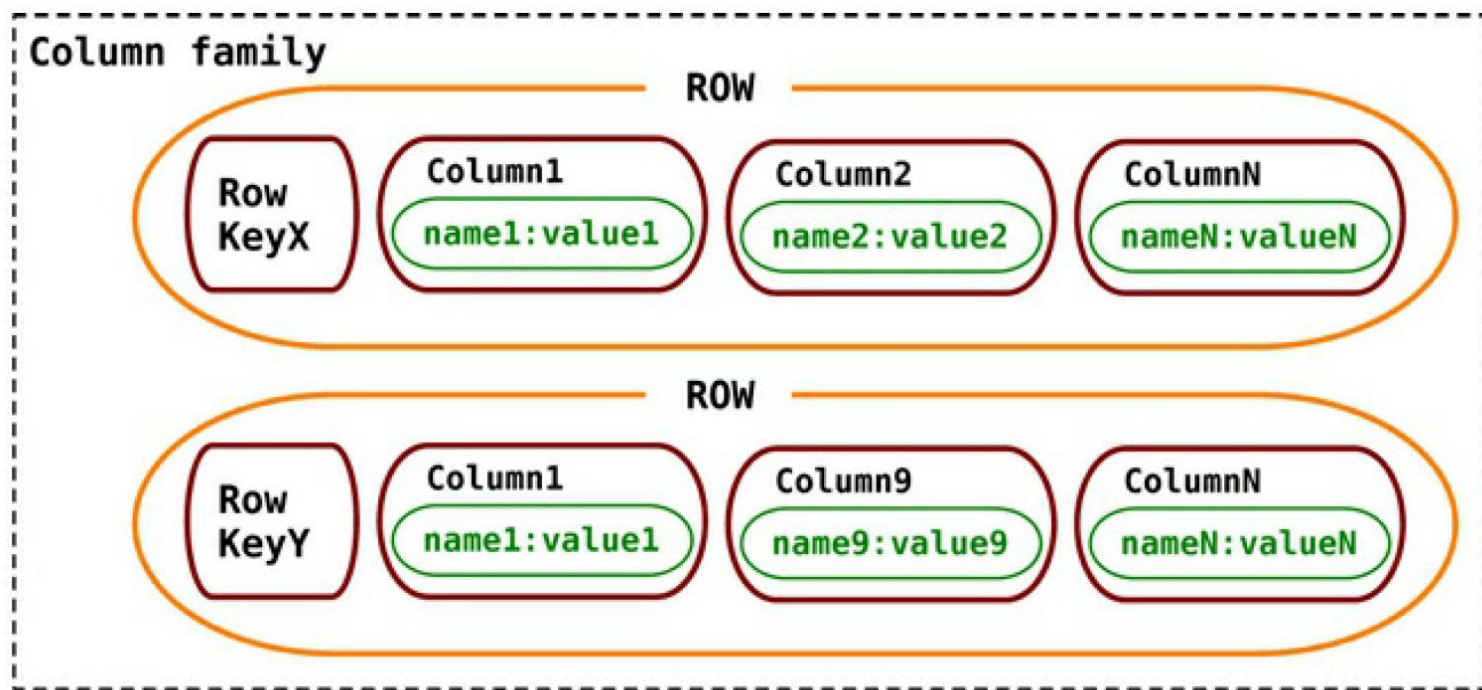


Figure 10.1. Cassandra's data model with column families

数据结构 1

- Cassandra的基本存储单元为“列”，列由一个“名值对”(name-value pair)组成，其中的名字也充当关键字。
- 每个键值对都占据一列，并且都存有一个“时间戳”值。令数据过期、解决写入冲突、处理陈旧数据等操作都会用到时间戳。若某列数据不再使用，则数据库可于稍后的“压缩阶段”(compaction phase)回收其所占空间。
- 行是列的集合，这些列都附在某个关键字名下，或与之相连。由相似行所构成的集合就是列族。
- 每个列族都可以与关系型数据库的“行容器”(container of rows)相对照：
 - 两者都用关键字标识行，并且每一行都由多个列组成。
 - 其差别在于，列族数据库的各行不一定要具备完全相同的列，并且可以随意向其中某行加入一列，而不用把它添加到其他行中。

数据结构 2

- “标准列族”(standard column family)中的列都是“简单列”(simple column)。
- “超列族”(super column family):
 - 如果某列中包含一个由小列组成的映射表, 那么它就是“超列”(super column)。可将超列视为“列容器”(container of columns)。
 - 用超列构建的列族叫做“超列族”。
 - 超列族适合将相关数据存在一起。但是, 如果部分列在大部分情况下都用不到, 则存在不必要的开销。
- “键空间”(keyspace)与关系型数据库中的“数据库”类似, 与应用程序有关的全部列族都存放于此。
 - 必须先创建键空间, 才能为其增添列族

一致性 1

- Cassandra收到写入请求后，会先将待写数据记录到“提交日志”(commit log)中，然后将其写入内存里一个名为“内存表”(memtable)的结构中。写入操作在写入“提交日志”及“内存表”后，就算成功了。
- 写入请求成批堆积在内存中，并定期写入一种叫做“SSTable”的结构中。该结构中的缓存一旦写入数据库，就不会再向其继续写入了。
- 若其数据变动，则需新写一张SSTable。
- 无用的SSTable可由“压缩”(compaction)操作回收。

一致性 2

- 若不关心数据是否陈旧，或是需要高效执行读取操作，那么可以将“一致性”设为ONE，以低级别的“一致性”执行读取操作。那么当Cassandra收到读取请求后，会返回第一个副本中的数据
 - 即便其是陈旧数据，也照样返回。
 - 如发现数据陈旧，则启动“读取修复”(read repair)过程
- 若需要极为高效的写入操作，并且不介意丢失某些写入的数据，那么可以将“一致性”设为ONE，以最低的“一致性”执行写入操作。那么Cassandra只将其写入一个节点的“提交日志”中，然后就向客户端返回响应。
 - 此时，如果某节点在尚未将写入的数据复制到其他节点前出了故障，那么这些数据就会丢失。

一致性 3

- 若将读取与写入操作的“一致性”都设为QUORUM
 - 那么读取操作将在过半数的节点响应之后，根据时间戳返回最新的列数据给客户端，并通过“读取修复”操作把最新数据复制到那些陈旧的副本中
 - 而“一致性”为QUORUM的写入操作则必须等所写数据传播至过半数的节点后，才能顺利结束其工作并通知客户端。
- 如果将“一致性”级别设为ALL，那么全部节点就必须响应读取或写入操作
 - 这将使集群失去容错能力：一旦某个节点故障，全部读取操作或写入操作都将阻塞并失败。
- 系统设计师应根据应用程序需求调整“一致性”级别，同一应用程序内部也会有不同的“一致性”需求，所以也可以针对每次操作来设定其“一致性”。
 - 例如，显示产品评论所需的“一致性”，就与读取客户所下最新订单状态不同

一致性 4

- 在创建“键空间”时，可以配置存储数据用的副本数，它决定了数据的“复制因子”。
- 若复制因子为3，则数据将复制至3个节点上。使用Cassandra写入及读取数据时，若将“一致性”设为2，则 $R+W$ 的值就会大于复制因子($2+2>3$)，这使得读取操作与写入操作的“一致性”都比较好。
- 可以在“键空间”上执行“节点修复”(node repair)命令，这会迫使Cassandra将其负责的每一个关键字与其余副本相比对。
 - 由于此操作开销较大，所以有时可以只修复一个或一组列族。
- 若某节点故障，则其存储数据会移交给其他节点。而当它重新上线时，数据库会把变更后的数据交还此节点。这种技术叫做“提示移交”(hinted handoff)，它可以帮助故障节点更快地恢复。

事务

- Cassandra没有传统意义上的“事务”（即封装多个写入操作并决定是否提交其数据变更的单元）。
- Cassandra 的写入操作在“行”级别是“原子的”
 - 根据某个给定的行键向行中插入或更新多个列，将算作一个写入操作，它要么成功，要么失败。
 - 写入操作首先会写在“提交日志”及“内存表”中，只有它向这两者写入数据后，才算顺利执行完
 - 假如某节点故障，稍后可根据“提交日志”将数据变更恢复至该节点中，这与Oracle数据库中的“重做日志”(redo log) 类似。
- 可用ZooKeeper等外部的“事务”程序库同步读写操作。还有Cages等程序库可把ZooKeeper形式的“事务”封装起来。

可用性 1

- 因为集群里没有主节点，其中每个节点地位等同。
- 在“一致性”与“可用性”之间做出明智的权衡。减少操作请求的“一致性”级别，即可提升集群“可用性”。
- $(R+W) > N$ 。
 - W是成功执行写入操作所需的最小节点数
 - R是顺利执行读取操作所需获取的最小应答节点数
 - N是参与数据复制的节点数。
 - 对于某定值N，可改变R与W的值，以调整“可用性”。

可用性 2

- 假设在10节点的Cassandra集群中，有一个复制因子为3的“键空间”(N=3)。
 - 如果R=2且W=2，那么 $(2+2) > 3$ 。在此情况下，若有一个节点故障，则不影响“可用性”，因为数据还可以从其他两个节点中获得。
 - 若W=2而R=1，则集群在两个节点故障时将无法写入，但仍可读取。
 - 若R=2而W=1，则集群在两个节点故障时仍可写入，但无法读取。
- “键空间”与“读/写操作”应该按照需求来设置：要么提高写入操作的“可用性”，要么提高读取操作的“可用性”。

查询功能 1

- 由于Cassandra没有功能丰富的查询语言，所以在设计其数据模型时，应该优化列与列族，以提升数据读取速度。
 - 在列族中插入数据后，每行中的数据都会按列名排序。
 - 假如某一列的获取次数比其他列更频繁，那么为了性能起见，应该将其值用作行键。
- 基本操作：
 - 指定“键空间”作为查询范围
 - 通过CREATE定义列族
 - 通过SET向列族插入数据或更新数据
 - 通过GET获取整个列族或列族所需的一列
 - 通过DEL删去一列或整个列族

查询功能 2

- 高级查询与索引编订：
 - Cassandra的列族可以用关键字之外的其他列当索引。
 - 然后直接通过索引进行查询
 - 索引以“位映射图”(bit-mapped) 的形式实现，在列中频繁出现重复数值的情况下，性能较好。
- Cassandra查询语言(CQL)，Cassandra支持一种类似SQL命令的查询语言，叫做“Cassandra查询语言”(Cassandra Query Language, 简称CQL)。
 - CQL命令可以创建列族，插入数据，读取读出全部列或者只读取需要的列，为列创立索引，并根据索引查询数据
 - CQL中还有很多查询数据的功能，不过它并未包含SQL的全部功能。CQL不支持“连接”(JOIN)及“子查询”(subquery)，而且其where子句通常也比较简单。

可扩展性

- 在已有的Cassandra集群中扩展，也就意味着增加更多节点。
- 由于不存在主节点，所以向集群中新增节点后，即可改善其服务能力，令其可以处理更多的写入及读取操作。
- 这种横向扩展可以尽力提高其正常运行时间，因为集群在新增节点时，仍能处理客户端请求。

适用案例-事件记录

- 由于列族数据库可存放任意数据结构，所以它很适合用来保存应用程序状态或运行中遇到的错误等事件信息。
- 在企业级环境下，所有应用程序都可以把事件写入Cassandra数据库。它们可以用appname: timestamp (应用程序名:时间戳) 作为行键，并使用自己需要的列。
- 由于Cassandra的写入能力可扩展，所以在事件记录系统中使用它效果会很好。

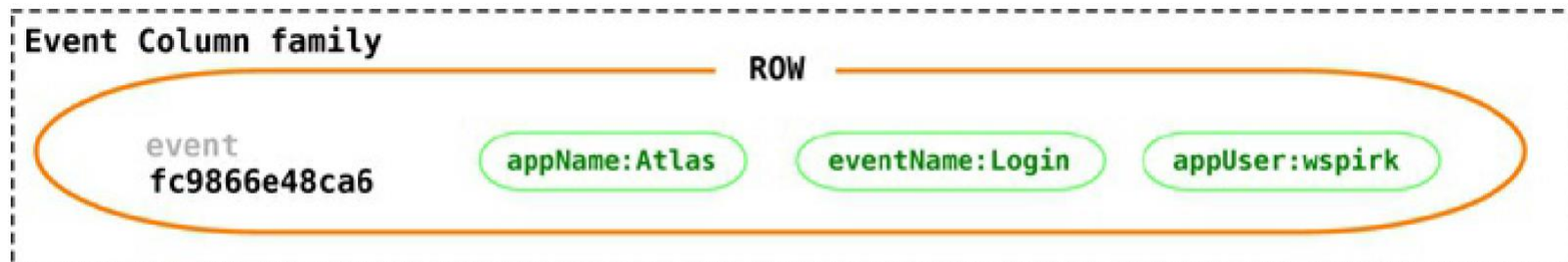


Figure 10.2. Event logging with Cassandra

适用案例-内容管理系统与博客平台

- 使用列族，可以把博文的“标签”(tag)、“类别”(category)、“链接”(link)和“trackback”日等属性放在不同的列中。
- 评论信息既可以与上述内容放在同一行，也可以移到另一个“键空间”。
- 同理，博客用户与实际博文亦可存于不同列族中。

适用案例-其他

● 计数器

- 在网络应用程序中，通常要统计某页面的访问人数并对其分类，以算出分析数据。此时可使用CounterColumnType来创建列族。
- 创建好列族后，可以使用任意列记录网络应用程序中每个用户访问每一页面的次数。
- 也可以用CQL增加计数器的值

● 限期

- 可能需要向用户提供试用版，或是在网站上将某个广告条显示一定时间。这些功能可以通过“带过期时限的列”(expiringcolumn)来完成。
- 这种列过了给定时限后，就会由Cassandra自动删除。这个时限叫做TTL (Time To Live, 生存时间)，以秒为单位。
- 经过TTL指定的时长后，这种列就被删掉了。程序若检测到此列不存在，则可收回用户访问权限或移除广告条。

不适用场合

- 需要以“ACID事务”执行写入及读取操作的系统。
- 如果想让数据库根据查询结果来聚合数据(例如SUM(求和)或AVG (求平均值)),那么得把每一行数据都读到客户端, 并在此执行操作。
- 在开发早期原型或刚开始试探某个技术方案时, 不太适合用Cassandra。开发初期无法确定查询模式的变化情况, 而查询模式一旦改变, 列族的设计也要随之修改。这将阻碍产品创新团队的工作并降低开发者的生产能力。
 - 在关系型数据库中, 数据模式的修改成本很高, 而这却降低了查询模式的修改成本
 - Cassandra 则与之相反, 改变其查询模式要比改变其数据模式代价更高。